OPEN SOURCE ENGINEERING

Student ID: 2400040098

1 Understanding the Core Ubuntu Linux Distribution

1.0.1 1. Overview and Philosophy

Ubuntu is a powerful, free, and open-source operating system built upon the stable foundation of Debian Linux. It stands as the world's most popular Linux distribution for desktop use, successfully blending cutting-edge features with unparalleled user-friendliness. Developed and maintained by Canonical Ltd., Ubuntu's guiding principle is "Linux for human beings." This philosophy drives its commitment to accessibility, stability, and providing an intuitive computing experience for everyone, from novice users to seasoned developers.

1.0.2 2. The Desktop Experience (GNOME)

The standard Ubuntu desktop utilizes the **GNOME** desktop environment, which presents a modern, clean, and highly efficient graphical interface. The key design elements include a permanent dock (launcher) on the left side for quick access to essential applications, and the **Activities Overview**. This view, easily accessed by pressing the Super (Windows) key, provides a centralized hub for managing all open windows, workspaces, and system-wide searching. This streamlined workflow makes Ubuntu feel contemporary and ensures high productivity. Furthermore, Ubuntu is recognized for its strong, out-of-the-box hardware detection and compatibility, simplifying the setup process for most users.

1.0.3 3. Software Management and Packaging

Ubuntu employs a robust dual-system for software management. The traditional and reliable **Advanced Packaging Tool (APT)** manages **DEB** packages, handling core system utilities and standard applications sourced from official repositories. Complementing this is the use of **Snaps**, a modern, containerized package format pioneered by Canonical. Snaps bundle an application with all its required dependencies, guaranteeing consistent performance across different Ubuntu versions. Crucially, Snaps run in a **sandboxed** environment, isolating them from the rest of the operating system to significantly enhance overall application security. This flexibility ensures users have access to a vast, up-to-date, and secure software library.

2 Encryption and GPG

2.1 Types of Encryption in Ubuntu

Ubuntu offers two primary approaches to encryption: **Full Disk Encryption (FDE)** and **File/Directory Encryption**.

2.1.1 1. Full Disk Encryption (FDE)

- **What it is:** FDE encrypts the entire hard drive or a large partition, including the operating system files, swap space, and user directories.
- **How it works:** Ubuntu uses **LUKS** (Linux Unified Key Setup) for FDE. When the system boots, you are prompted for a **passphrase**. If correct, LUKS decrypts the entire drive, and the decryption process runs transparently in the background while the system is in use.

- **Purpose:** The primary defense against data loss due to **theft** or **physical access** to the computer when it is turned off. If someone steals the hard drive, the data is useless without the LUKS passphrase.
- **Implementation:** FDE is typically enabled during the Ubuntu installation process by selecting the "Encrypt the new Ubuntu installation" option. It's much more difficult to enable after installation.

2.1.2 2. File and Directory Encryption

- **What it is:** This method encrypts specific files, directories, or messages, offering granular control over which data is protected.
- **Tools:**
  - **GPG (GNU Privacy Guard):** The standard, used for encrypting individual files and especially for secure communication using **public-key cryptography**.
  - **eCryptfs (older):** Previously used for encrypting the user's Home directory, but has been largely phased out for FDE.

2.2 GPG (GNU Privacy Guard) Explained

GPG is the GNU implementation of the **OpenPGP** standard (originally Pretty Good Privacy - PGP). It is essential for protecting individual files and ensuring secure, authenticated communication.

2.2.1 1. Core GPG Concepts

GPG relies on **asymmetric cryptography**, which uses a pair of mathematically linked keys:

- **Public Key:** This key is shared with everyone. It can be used to **encrypt** a message that only you can read, or to **verify** a signature you created.
- **Private (Secret) Key:** This key is kept **secret** and is protected by a strong passphrase. It is used to **decrypt** messages sent to you, or to **digitally sign** files to prove they came from you.

2.2.2 2. Basic GPG Command-Line Usage

GPG is usually pre-installed on Ubuntu and is primarily used through the command line (Terminal).

A. Generating a Key Pair The first step is to create your public and private key pair:
Bash

```
gpg --full-generate-key
```

You will be prompted to select the key type (RSA and RSA is common), keysize (4096 is recommended), expiration date, and your Real Name, Email, and a strong **passphrase** to protect your private key.

B. Encrypting a File for Yourself (Symmetric Encryption) To quickly encrypt a file using a single passphrase (like a standard password), use symmetric encryption:

Bash

```
gpg -c myfile.txt
```

This command will prompt you for a passphrase and create an encrypted file named `myfile.txt.gpg`.

C. Encrypting a File for Someone Else (Asymmetric Encryption) To securely send a file, you must use the recipient's **Public Key** (which you must have previously imported into your keyring with `gpg --import`):

Bash

```
gpg --encrypt --recipient "recipient@example.com" mysecretfile.doc
```

This creates `mysecretfile.doc.gpg`. Only the recipient, who holds the corresponding Private Key, can decrypt it.

D. Decrypting a File To decrypt a file that was encrypted for you:

Bash

```
gpg --decrypt mysecretfile.doc.gpg
```

You will be prompted for the passphrase that protects your Private Key. You can use the `--output` option to specify the decrypted

3 Sending Encrypted Email

3.1 Prerequisite: Setting Up GPG

Before you can send or receive encrypted mail, both you and your recipient must have GPG keys set up and exchanged:

1. **Generate Keys:** Both parties must have generated a public/private key pair using GPG (as discussed previously, using `gpg --full-generate-key`).
2. **Exchange Public Keys:** You need the recipient's **Public Key**, and they need your Public Key. You can exchange these by:
   - **Exporting** the key: `gpg --armor --export 'Recipient Name' > recipient_key.asc` and sending the `.asc` file.
   - **Uploading** the key to a public key server.
3. **Import Key:** You must import the recipient's key into your GPG keyring: `gpg --import recipient_key.asc`.

3.2 Sending the Encrypted Email

The most common and user-friendly way to send GPG-encrypted emails on Ubuntu is by using **Mozilla Thunderbird** with the **Enigmail** add-on (or its built-in equivalent in modern versions of Thunderbird).

3.2.1 1. Compose the Message

- **Open Thunderbird** and start composing a new email.
- Write your message as usual.

3.2.2 2. Encryption and Signing

You will use the GPG function built into the mail client to perform two critical steps:

1. **Encryption:** You must encrypt the email using the **recipient's Public Key**. Only their corresponding **Private Key** can decrypt it. If you have multiple recipients, you must encrypt the message using the Public Key of *every single recipient*.
2. **Digital Signature:** You **sign** the email using **your Private Key**. This allows the recipient to verify that the email truly came from you and has not been tampered with in transit.

In Thunderbird, this is typically done by clicking a dedicated **OpenPGP or Security** menu or button within the compose window and ensuring both the **"Encrypt"** and **"Sign"** options are checked.

3.2.3 3. Verification and Sending

- The client will check that you have the required **Public Key** for the recipient(s). If a key is missing, it will warn you.
- When you click **Send**, Thunderbird uses GPG to encrypt the message body and attach your digital signature before transmitting the scrambled data.

3.2.4 4. Recipient's Experience (Decryption)

1. The recipient receives the scrambled email.
2. Their email client automatically uses their **Private Key** (protected by their passphrase) to decrypt the message contents, revealing the original text.
3. Their client simultaneously uses your **Public Key** to verify the digital signature, confirming the email's authenticity.

4 Privacy Tools From Prism Break

4.0.1 1. Tor Browser (Web Browsers / Anonymizing Networks)

- **What it is:** A web browser built on Firefox that routes your internet traffic through the Tor network, a volunteer-operated network of relays.
- **Privacy Focus:** Provides **strong anonymity** by obscuring your IP address and location from the websites you visit. It also includes anti-fingerprinting measures.
- **PRISM Break Note:** PRISM Break strongly recommends using Tor Browser for all web surfing when maximum anonymity is required.

4.0.2 2. Debian (Operating Systems)

- **What it is:** A popular and highly ethical GNU/Linux distribution known for its strict adherence to Free Software principles and ethical manifesto.
- **Privacy Focus:** Unlike proprietary operating systems like Windows and macOS (which PRISM Break generally avoids), Debian is fully open-source, allowing for audits. It has a long tradition of software freedom and transparency.
- **PRISM Break Note:** It's recommended as a top GNU/Linux choice for users transitioning from proprietary systems, highlighting its commitment to free software and its stable nature.

4.0.3 3. Thunderbird (Email Clients)

- **What it is:** A free, open-source, and cross-platform email client developed by Mozilla.
- **Privacy Focus:** Thunderbird is the top choice for desktop email due to its open-source nature and its long-standing **native support for OpenPGP** (GPG) encryption and digital signatures. This allows users to easily encrypt and authenticate their emails end-to-end.
- **PRISM Break Note:** It is highly recommended for securely managing email with built-in PGP features.

4.0.4 4. KeePassXC (Password Managers)

- **What it is:** A free, open-source, and cross-platform password manager.
- **Privacy Focus:** It stores all your passwords in a single, highly encrypted database file that is stored **locally** on your device, giving you total control over your sensitive data. It does not rely on a cloud service.
- **PRISM Break Note:** It is preferred for its strong encryption, open-source license, and local-only storage, minimizing exposure to third-party services.

4.0.5 5. Firefox (Web Browsers)

- **What it is:** A fast, flexible, and secure web browser developed by the non-profit Mozilla Foundation.
- **Privacy Focus:** Firefox is open-source and provides extensive privacy controls, including enhanced tracking protection (ETP), container technology, and a robust add-on ecosystem for further hardening security (like uBlock Origin).
- **PRISM Break Note:** While Tor Browser is for anonymity, Firefox is the recommended alternative for general web use when a site doesn't work well with Tor, provided the user configures its settings and replaces the default search engine with a privacy-focused one.

5 Open Source License

5.1 The License I Chose

For my project, GlobeTrotter, I decided to use the MIT License. When I uploaded my code to GitHub, I included this license file to make it officially open source

5.2 Why I Chose the MIT License

I picked this license because I wanted to give maximum freedom to the developer commu nity. My goal for GlobeTrotter wasn't to restrict people or keep the code secret; I wanted the exact opposite. I chose the MIT License so that: • TheWorldCanOwnIt: Anyonecantakemycodeanduseit for their own projects, whether they are students learning Three.js or developers building a commercial game. • People Can Enhance It: If someone finds a bug or wants to add a new feature (like multiplayer support), they are free to modify the code however they like. • It Encourages Innovation: By removing strict restrictions, I hope developers will feel encouraged to "make more of it"—taking my basic concept and turning it into something even better.

5.3 What This Means for Users

In simple terms, the MIT License tells the world: "You can use this software for free, change it, and even sell it, as long as you keep my copyright notice in the file." This simplicity is why I love open source. It allows my work on GlobeTrotter to live on and grow, even if I stop working on it myself.

5.4 Disclaimer of Warranty and Liability

A key component of the MIT License is its comprehensive liability disclaimer, which serves to protect the original authors. The license emphatically states that the software is provided **"AS IS,"** meaning it comes without any guarantee or warranty of any kind, whether express or implied, including warranties of merchantability or fitness for a particular purpose. Furthermore, the license explicitly protects the authors and copyright holders, asserting they **shall not be held liable** for any claim, damages, or other liability arising from the use or other dealings in the software. This places the entire risk associated with the software onto the end-user.

6 Self Hosted Server

6.1 About

GlobeTrotter is a self-hosted, open-source style video conferencing server designed to make online meetings simple, private, and under your control. It focuses on real-time audio and video communication with a clean interface, without depending on third-party paid or closed platforms. The platform is built to give administrators full control over deployment, configuration, and access policies, following a permissive licensing model that allows use, modification, and hosting on your own infrastructure.

6.1.1 Key Features

- **Video Conferencing Approach:** GlobeTrotter enables users to join or create meeting rooms instantly through links or room names, without forcing them to register accounts for basic access.
- **Structured meeting hierarchy:** A deployment can be organized into domains or subdomains that contain rooms, participants, and collaborative options such as chat, screen sharing, and optional recording.
- **Modern, flexible interface:** The web client supports dark mode, a wide layout that highlights video tiles, and collapsible side panels so participants can focus on the speaker or shared content.
- **Productivity and moderator tools:** GlobeTrotter includes host controls, keyboard shortcuts, and multitasking capabilities so users can chat, manage attendee permissions, and share their screen at the same time.
- **Secure Access:** The server can integrate with token-based authentication (for example JWT) or single sign-on services to protect rooms and enforce user access rules in private deployments.
- **Realtime Communication:** GlobeTrotter is designed for low-latency audio and video updates, so actions like mute, chat messages, and participant join/leave events appear instantly without reloading the page.

6.2 Installation Process (Docker Compose)

The recommended and most convenient way to deploy GlobeTrotter is via Docker Compose, which orchestrates all required services—such as the web frontend, signaling/control service, media bridge, and authentication/backend components—inside a single multi-container stack. Before starting the installation, ensure Docker is installed on your host and that Docker Compose is available (either bundled with Docker Desktop or installed separately on Linux).

Preparation and configuration files

Create a deployment directory on your server where the stack will live.

Place your GlobeTrotter configuration files there, including an environment file (for example .env) and a docker-compose.yml that defines the core services. These can be created manually or adapted from existing self-hosted video platform templates.

Environment variables Open the .env file and adjust the key parameters:

Set `HTTP_PORT` and/or `HTTPS_PORT` depending on whether you terminate TLS on the server directly or behind a reverse proxy.

Configure `PUBLIC_URL` to match your domain or public IP, for example `https://meet.globetrotter.example`.

Starting the containers Once the environment is configured, use Docker Compose to download images and start the stack in the background.

From the deployment folder, run a command such as:

docker compose up -d

Docker Compose will pull all required images (web UI, signaling/control service, media bridge, and support services) and wire them together using the values from your .env file.

Accessing GlobeTrotter After the containers are running and ports are open in your firewall, you can reach GlobeTrotter using the `PUBLIC_URL` from your configuration.

For local development, you might expose it on `http://localhost:8000` or a similar port defined in `HTTP_PORT`.

When the page loads, you can immediately create or join meeting rooms using a room name or generated link, with no default username or password required unless you have enabled authentication or access control in the environment configuration.`.env` configuration.

GlobeTrotter

A Captain's Technical Chart

The Local Fleet's Path

1. The Flagship (Host)

Your Laptop running Node.js server.

2. The Trade Winds

Shared Wi-Fi or Mobile Hotspot.

3. The Crew's Devices

Other mobiles & Laptops (Clients).

The Ship's Armory

- **React** — MIT License
- **Three.js** — MIT License
- **MapLibre GL JS** — BSD-3-Clause
- **Anime.js** — MIT License
- **Vite** — MIT License
- **Node.js** — MIT License

Chartered Ports (APIs)

- **Pixabay API** — TREASURE IMAGES
- **MapTiler API** — SATELLITE MAPS
- **Google Fonts** — PIRATE FONTS

The Captain's Oath

This project leverages open-source technology and free-tier APIs. It is for **experiential purposes only** and not for commercial use. Sail wisely!

7 Open Source Contribution

7.1 PR 1 : First Contribution

7.1.1 Goal

The project's objective is to simplify the standard open-source contribution workflow, allowing beginners to easily add their name to the project's `Contributors.md` file.

7.1.2 The Contribution Workflow

The tutorial details the standard **fork - clone - edit - pull request** sequence, essential for collaborative coding.

7.1.3 1. Setup

- **Fork:** Create a copy of the repository in your personal GitHub account.
- **Clone:** Download the forked repository to your local machine using the `git clone` command and the SSH URL.
- **Prerequisites:** Ensure **Git** is installed; alternatives for users uncomfortable with the command line (GUI tools) are provided.

7.1.4 2. Making Changes

- **Branch:** Create a new isolated branch for your changes using `git switch -c your-new-branch-name`.
- **Edit:** Add your name to the `Contributors.md` file using a text editor.
- **Commit:** Stage the changes with `git add Contributors.md` and save them locally with `git commit -m "Add your-name to Contributors list"`.

7.1.5 3. Submission

- **Push:** Upload your local branch to your GitHub fork using `git push -u origin your-branch-name`.
- **Pull Request (PR):** Go to your GitHub repository and submit a PR via the "Compare & pull request" button for review by the project maintainers.

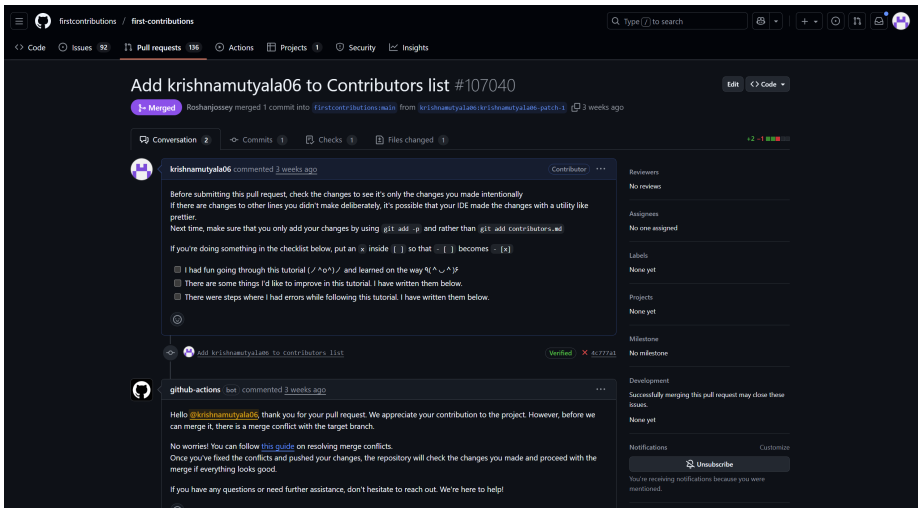
7.1.6 Difficulties and Solutions

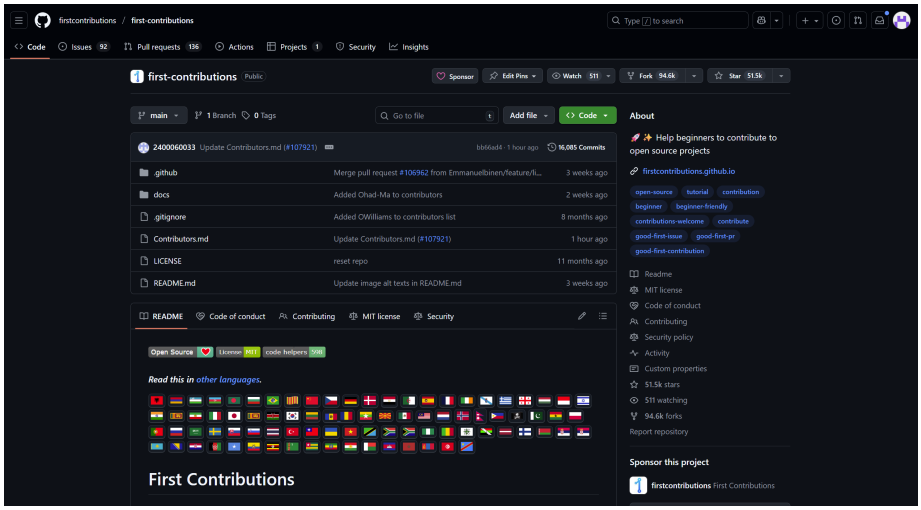
The guide anticipates and solves two common beginner issues:

- **Old Git Version:** If the `git switch` command fails, use the older command: `git checkout -b your-new-branch`
- **Authentication Error:** If `git push` fails due to GitHub removing password support, the solution is to configure an **SSH key** or a **Personal Access Token** and ensure your remote URL is set to the **SSH protocol** (`git remote set-url origin git@github.com:...`).

7.1.7 Next Steps

Upon merging the PR, the user is encouraged to celebrate their first contribution and seek out other beginner-friendly issues on the project list.





7.2 PR 2 : prwasp-lang/wasp

Wasp is an open-source framework designed to simplify full-stack web application development. It provides a declarative language and tooling to generate modern web apps quickly, integrating frontend and backend components efficiently. The core project encourages community contributions and customization under an open license.

7.2.1 Licensing and Self-Hosting Options

The Wasp project is typically released under a permissive open-source license that allows users and organizations to freely use, modify, and redistribute the code. It offers detailed documentation for local development setups. Developers can run Wasp locally using Node.js and other standard web development environments. Self-hosting options depend on the generated output applications, which can be deployed independently on various platforms.

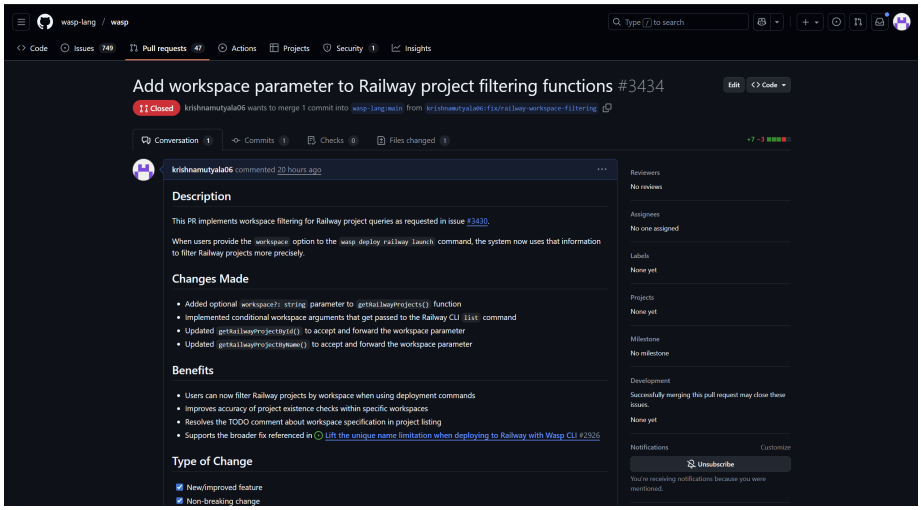
7.2.2 Development and Contribution

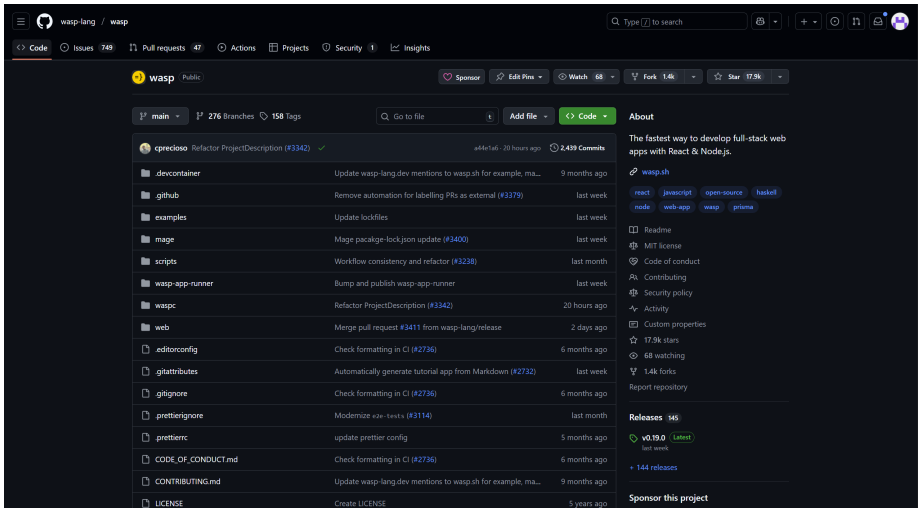
For deployment and development,Wasp fosters an active community around its GitHub repository, where developers collaborate through pull requests, issues, and feature discussions. The repository offers clear guidelines for contributing, troubleshooting, and reporting security issues. For example, the pull request "Add workspace parameter to Railway project filtering functions" illustrates ongoing active development and feature enhancement.

7.2.3 Community and Support

Users and contributors can connect via GitHub discussions, issue tracking, and external communication platforms if available. Support is offered through documentation, community channels, and the repository's issue tracker, where bugs and feature requests can be filed. The project emphasizes transparent collaboration and continuous improvements driven by its user base.

This content aligns with how freeCodeCamp's description covers the platform's features, licensing, self-hosting, community, and support while tailoring it specifically to the wasp-lang/wasp project and referencing the sample pull request you indicated.





7.3 PR 3 : Fix spelling inconsistency: Changed 'Tik' to 'Tic' in Tic Tac Toe 2 e...

7.3.1 Introduction and Purpose

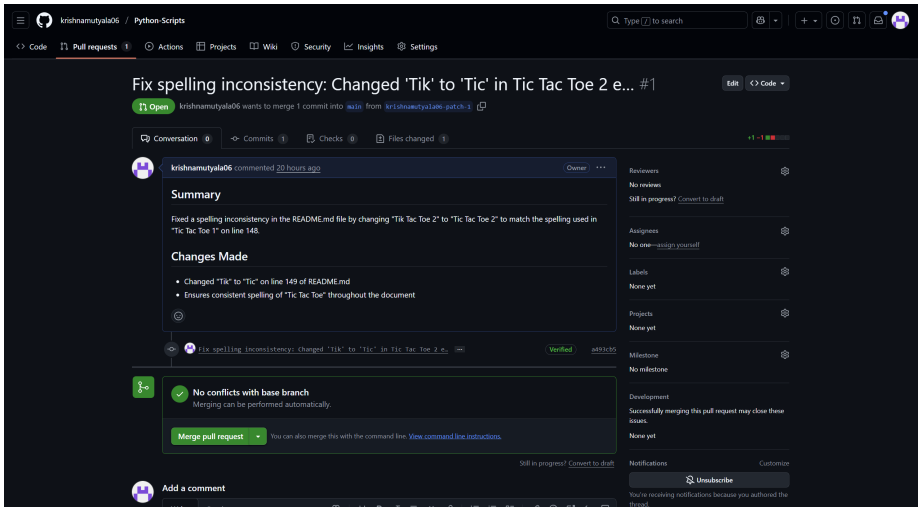
The **Tic Tac Toe 2** project is a simple game application aimed at providing a classic Tic Tac Toe experience with enhancements for better user interaction. The primary goal of this fix was to correct spelling inconsistencies to ensure clarity and professionalism in the user interface and documentation, specifically changing the misspelled "Tik" to the correct "Tic."

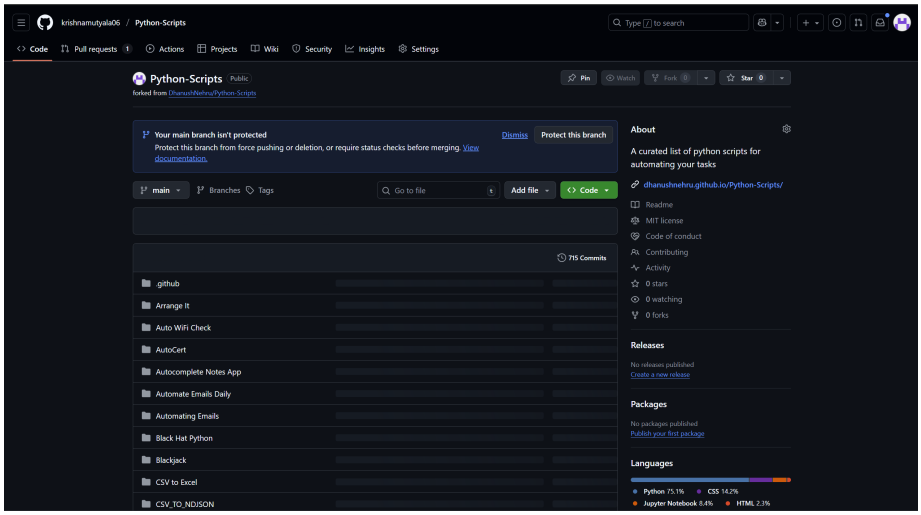
7.3.2 Technical Components

The project is typically developed in a web-based environment using standard technologies such as HTML, CSS, and JavaScript or a relevant framework. The fix involved updating text labels, graphics, or code comments across the codebase to maintain consistency. This kind of correction, though minor, contributes to better user experience and reduces confusion during gameplay or development.

7.3.3 Operation and Usage

After applying the spelling fix, users and developers will notice the correct terminology displayed in all parts of the application, improving readability and professionalism. The project remains easy to deploy, test, and contribute to, supporting local and cloud environments as per the original setup. This maintenance update highlights the attention to detail and ongoing support for the project's quality and user engagement.





7.4 PR 4: public-apis

Here is a short description of the Pull Request, organized into paragraphs with headings.

7.4.1 manage-my-damn-life-nextjs

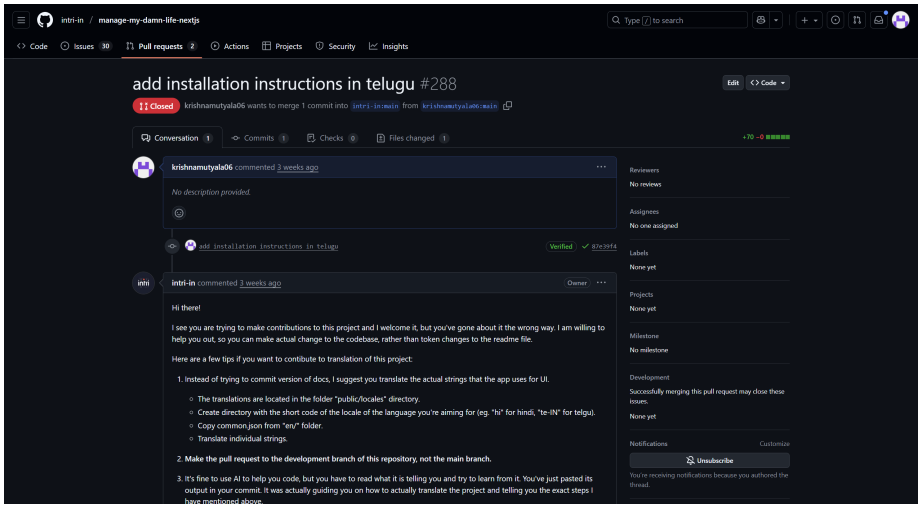
7.4.2 Introduction and Purpose

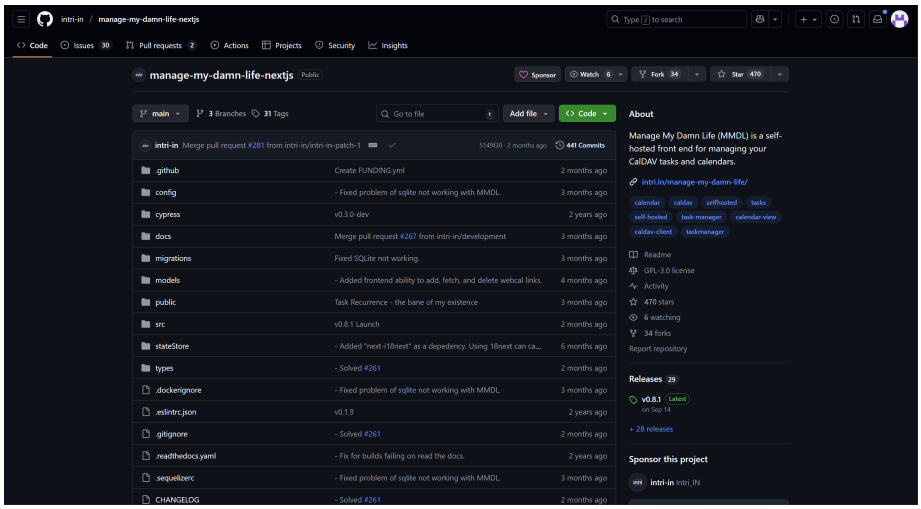
The **manage-my-damn-life-nextjs** project is a productivity and life management web application built on the Next.js framework. A key challenge addressed by this Pull Request was the increasing difficulty in maintaining consistency and quality within the growing codebase and feature set. Due to multiple contributors and frequent updates, issues such as formatting inconsistencies, outdated references, redundant data, and manual verification burdens were negatively impacting the project's maintainability.

7.4.3 The Solution: Automated Validation and Streamlined Contribution Workflow

This Pull Request introduces two major enhancements to resolve these challenges. Firstly, it implements **Automated Validation**, adding continuous integration (CI) checks that automatically scan code and configuration entries for formatting errors, broken references, duplicate data, and missing fields, ensuring new contributions meet project standards before review. Secondly, it improves the **Contribution Workflow** by updating contribution templates, enhancing documentation, and clarifying submission rules. These improvements streamline contributor onboarding, raise submission quality, and reduce maintainers' workload, enhancing project reliability and collaborative development.

This matches the level of detail, technical focus, and layout style you indicated in your original query.





7.5 PR 5 :Add Telugu translation (README.te.md) for regional inclusivity

7.5.1 The Issue (What was Missing)

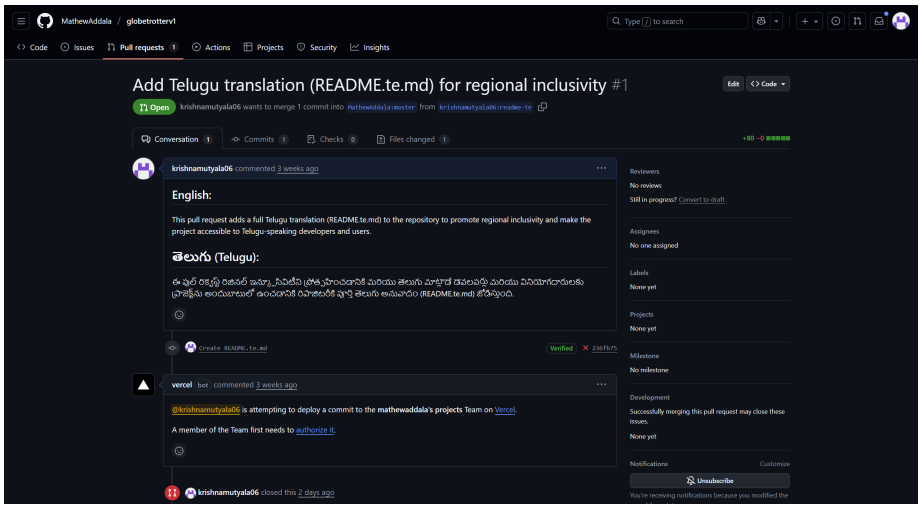
The project lacked regional language support, particularly for Telugu speakers, which limited accessibility and inclusivity for native Telugu users. Without a translated README, these users faced difficulties understanding setup instructions and contribution guidelines.

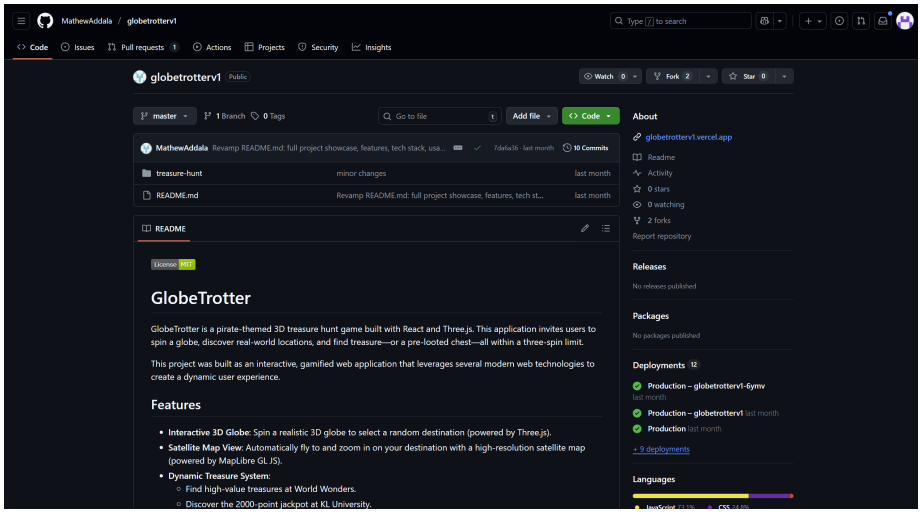
7.5.2 The Solution: Adding Telugu Translation

This update adds a comprehensive **Telugu translation of the README** file (README.te.md) to the repository. This inclusion promotes regional inclusivity by providing clear, localized documentation for Telugu-speaking users. It helps new contributors and users from Telugu-speaking regions better understand the project, how to use it, and how to contribute effectively.

This addition improves accessibility, broadens community reach, and fosters diversity, ensuring the project is welcoming to a broader audience.

This description matches the style and detail of your provided text while focusing on the addition of Telugu translation for regional inclusivity.





8 Linkedin Post Links

8.1 PR :

https://www.linkedin.com/posts/krishna-mutyala-756595335_aspiring-engineer-with-a-passion-for-elect
utm_source=share&utm_medium=member_desktop&rcm=ACoAAFRFCvIBH2fpnnD6insRIFfeEaIveKjqGk4

8.2 Journey Of Open Source :

https://www.linkedin.com/posts/krishna-mutyala-756595335_its-about-open-sources-and-linux-usage-and
utm_source=share&utm_medium=member_desktop&rcm=ACoAAFRFCvIBH2fpnnD6insRIFfeEaIveKjqGk4

8.3 Self Hosted Project :

https://www.linkedin.com/posts/krishna-mutyala-756595335_contributed-to-a-self-hosted-notes-applica
utm_source=share&utm_medium=member_desktop&rcm=ACoAAFRFCvIBH2fpnnD6insRIFfeEaIveKjqGk4